

Test technique

Mission développement IA — Durée annoncée : 2h maximum

Ne passez pas plus de temps que 2h sur cet exercice. Nous évaluons votre approche et votre rigueur, pas la complétude à 100%.

Deadline : 3 jours à compter de la réception de ce document.

Contexte fictif (à des fins d'évaluation uniquement) : vous devez construire un service qui génère des questions à choix multiples sur un sujet donné, en utilisant un LLM (OpenAI ou Anthropic), avec un niveau de difficulté paramétrable.

Partie 1 — Construction (obligatoire)

Construisez un endpoint (Node.js/TypeScript, Cloudflare Workers ou Express, au choix) qui :

1. Reçoit en entrée : un sujet (string, ex: "géographie mondiale"), un niveau (enum: "facile" | "moyen" | "difficile"), et un nombre_questions (number, 1 à 10).
2. Appelle un LLM (OpenAI ou Anthropic) pour générer les questions.
3. Retourne une réponse conforme au schéma Zod suivant :

```
import { z } from "zod";

const QuestionSchema = z.object({
  question: z.string(),
  options: z.array(z.string()).length(4),
  bonne_reponse: z.number().min(0).max(3), // index de la bonne option
  explication: z.string(),
});

const ReponseSchema = z.object({
  sujet: z.string(),
  niveau: z.enum(["facile", "moyen", "difficile"]),
  questions: z.array(QuestionSchema),
});
```

4. Implémentez une logique de retry : si la sortie du LLM ne respecte pas le schéma (JSON malformé, champs manquants, mauvais type), relancez l'appel avec un prompt ajusté, jusqu'à 3 tentatives. Si les 3 tentatives échouent, retournez une erreur explicite.
5. Ajoutez au moins un test automatisé (Vitest, Jest, ou équivalent) qui vérifie que la réponse respecte le schéma sur un cas nominal, et un test qui vérifie le comportement du retry sur une réponse malformée simulée (mock).

Livrable : un dépôt Git (public temporaire, ou zip) avec le code, un README.md expliquant vos choix techniques, et les tests.

Partie 2 — Debug et esprit critique (obligatoire)

Voici un extrait de code et le prompt qu'il utilise. Il contient 3 problèmes de conception (pas de simples bugs de syntaxe — de vrais problèmes d'architecture ou de fiabilité). Identifiez-les et proposez une correction pour chacun.

```
async function genererQuestion(sujet: string): Promise<any> {
```

```
const prompt = `Génère une question de quiz sur ${sujet}. Réponds en JSON.`;

const response = await openai.chat.completions.create({
  model: "gpt-4",
  messages: [{ role: "user", content: prompt }],
});

const data = JSON.parse(response.choices[0].message.content);
return data;
}

async function genererDixQuestions(sujet: string) {
  const questions = [];
  for (let i = 0; i < 10; i++) {
    questions.push(await genererQuestion(sujet));
  }
  return questions;
}
```

Indice : pensez fiabilité de la sortie, performance/coût, et robustesse face aux erreurs.

Comment nous évaluons

- La qualité du raisonnement compte plus que la complétude du code.
- Nous discuterons de vos choix en entretien si vous êtes sélectionné-e parmi les finalistes — préparez-vous à expliquer en détail pourquoi vous avez fait tel ou tel choix.
- Merci de nous indiquer, dans votre README, qui dans votre équipe a travaillé sur quelle partie si vous êtes plusieurs.

Bonne chance ! Merci de renvoyer votre livrable dans les 3 jours à lea@gofrench-academy.com